CentraleSupélec

Mathematics and Data Science

Research Project Report

Academic Year 2025-2026

Probabilistic Detection of Market Manipulation

Adonis JAMAL Jean-Vincent MARTINI

Supervisor: Damien CHALLET

Academic Tutor: Lionel GABET

April 2026

Abstract

https://github.com/adonis107/MDS-PDMM

Acknowledgements

Contents

1 Introduction **5**
1.1 Context and Motivation . . . 5
1.2 Research Questions . . . 6
1.3 Contributions . . . 6
1.4 Report Outline . . . 7

2 Background and Related Work **8**
2.1 Limit Order Books and Market Microstructure . . . 8
2.2 Market Manipulation Strategies . . . 8
2.3 Anomaly Detection in Financial Data . . . 9
2.3.1 Reconstruction-Based Approaches . . . 9
2.3.2 Probabilistic Scoring via Synthetic Manipulation . . . 10
2.3.3 One-Class and Density-Based Approaches . . . 10
2.4 Thresholding Methods for Anomaly Detection . . . 11
2.5 Feature Attribution Methods . . . 11

3 Methodology **13**
3.1 Feature Engineering . . . 13
3.1.1 Order Book Imbalance . . . 13
3.1.2 Price Dynamics and Microstructure . . . 14
3.1.3 Liquidity and Elasticity . . . 15
3.1.4 Volatility . . . 15
3.1.5 Event Flow and Rapidity . . . 16
3.1.6 Hawkes-Inspired Memory Features . . . 16
3.1.7 Order Flow Imbalance . . . 18
3.1.8 Preprocessing and Scaling . . . 18
3.2 Detection Models . . . 19
3.2.1 Transformer Bottleneck Autoencoder with One-Class SVM . . . 19
3.2.2 Probabilistic Neural Network . . . 22
3.2.3 Probabilistic Robust Autoencoder . . . 24
3.3 Anomaly Thresholding . . . 26
3.3.1 Peak Over Threshold . . . 26
3.3.2 SPOT and DSPOT . . . 26
3.3.3 Rolling False Discovery Rate . . . 26
3.4 Spoofing Gain Estimation . . . 26
3.5 Interpretability . . . 26
3.5.1 Integrated Gradients . . . 26
3.5.2 Grouped Occlusion . . . 26
3.6 Post-Hoc Analysis . . . 26
3.6.1 Anomaly Clustering . . . 26

3.6.2 Jump Analysis . . . 26

4 Data and Experimental Setup **27**
4.1 Dataset Description . . . 27
4.2 Preprocessing Pipeline . . . 27
4.3 Training Protocol . . . 27
4.3.1 Model Hyperparameters . . . 27
4.4 Test Protocol . . . 27
4.5 Compute Environment . . . 27

5 Results **28**
5.1 Model Diagnostics . . . 28
5.1.1 Transformer + OC-SVM . . . 28
5.1.2 PNN . . . 28
5.1.3 PRAE . . . 28
5.2 Anomaly Detection on Test Data . . . 28
5.2.1 Anomaly Rates by Model and Time Period . . . 28
5.2.2 Score Time Series . . . 28
5.2.3 Model Agreement . . . 28
5.3 Threshold Comparison . . . 28
5.4 Spoofing Gain Analysis . . . 28
5.5 Sensitivity and Feature Attribution . . . 28
5.5.1 Integrated Gradients: PNN and PRAE . . . 28
5.5.2 Grouped Occlusion: Transformer + OC-SVM . . . 28
5.5.3 Cross-Model Feature Importance . . . 28
5.6 Anomaly Clustering . . . 28
5.7 Jump Analysis . . . 28
5.8 Cross-Year Robustness . . . 28

6 Discussion **29**
6.1 Limitations . . . 29
6.2 Comparison of Detection Approaches . . . 29
6.3 Interpretation of Detected Anomalies . . . 29

7 Conclusion **30**
7.1 Summary of Contributions . . . 30
7.2 Future Work . . . 30

A Feature Catalogue **32**

B Hyperparameter Tables **33**

C Additional Figures **34**
C.1 Score Distributions per Test Day . . . 34
C.2 LOB Snapshots for Selected Anomalies . . . 34
C.3 Feature Evolution Around Anomalies . . . 34

D Threshold Method Details **35**

E Spoofing Gain Derivation **36**

Chapter 1

Introduction

1.1 Context and Motivation

Modern securities exchanges organise trading through electronic limit order books. A limit order book (LOB) records all outstanding buy and sell orders at each price level, providing a real-time, public summary of supply and demand. The instantaneous state of the book, defined by the distribution of volume across price levels on both sides, and the recent flow of incoming orders jointly determine short-term price dynamics. Tao et al. [7] model this dependence through a multi-level volume imbalance that aggregates supply and demand across several price levels, showing that imbalance shifts are directly linked to the probability distribution of subsequent price moves. Fabre and Challet [1] extend this line of work by introducing Hawkes-inspired order flow features that capture both the size and the distance of placement of new limit orders, demonstrating on cryptocurrency data that these features predict the conditional distribution of mid-price movements.

Market manipulation exploits this dependence. By artificially altering either the instantaneous book state or the order flow, a manipulator can induce a predictable price response and profit from it. Spoofing, the primary focus of this project, consists of placing large visible limit orders with no intention of execution. These orders shift the perceived supply-demand balance, move the mid-price in the desired direction, and are cancelled once the manipulator's genuine order on the opposite side has been filled. Layering extends spoofing by stacking multiple non-bona-fide orders across several price levels, amplifying the perceived depth on one side of the book [1]. Both strategies distort the volume imbalance: Tao et al. [7] show that the optimal spoofing action shifts the multi-level imbalance away from its equilibrium value, and that comparing the observed imbalance before a market order to its long-run distribution provides a basis for detection.

The regulatory treatment of these strategies differs sharply between asset classes. In regulated equity and derivatives markets, spoofing and layering are explicitly prohibited under the European Market Abuse Regulation (EU MAR) and the US Dodd-Frank Act. In cryptocurrency markets, the absence of equivalent regulation opens the door to manipulative behaviour on a larger scale [1]. Fabre and Challet, working on Level-3 crypto order book data, find that 31% of large orders on the BTC-USD pair could profitably spoof the market over a four-day period. This asymmetry suggests that crypto order book data should contain a substantially higher base rate of manipulative patterns than equity data, a hypothesis that motivates future comparative work.

A binary classifier that labels each event as "manipulative" or "not" is poorly suited to this detection problem for three reasons. First, no ground-truth labels exist: in equity markets,

confirmed manipulation cases are rare and not publicly annotated at the order level. Second, manipulation is not a discrete category but a spectrum of intent, from aggressive but legitimate liquidity provision to unambiguous spoofing. Third, a probabilistic score is more actionable than a binary flag: it allows a surveillance system to rank events by severity and calibrate alert thresholds to a desired false discovery rate. Fabre and Challet [1] address these requirements with a probabilistic neural network that outputs a full predictive distribution over future price moves, from which both an anomaly score (the negative log-likelihood) and an economic spoofing gain estimate can be derived.

1.2 Research Questions

This project investigates the following questions:

1. Can unsupervised probabilistic models trained on order book features detect anomalous order flow in the absence of ground-truth labels?
2. Which order book features carry the most information about anomalous events, and do different detection models agree on the same discriminative features?
3. How should anomaly thresholds be calibrated in a non-stationary, streaming setting when no labelled data are available?
4. Can the economic profitability of a spoofing strategy be estimated directly from model predictions, providing an interpretable measure of manipulation opportunity?
5. Do detection patterns remain stable across different time periods, when models trained on one year are applied to data from other years?

These questions are addressed on the TOTF.PA (TotalEnergies, Euronext Paris) limit order book dataset, covering trading days from 2009, 2010, 2015, and 2017.

1.3 Contributions

The main contributions of this report are:

1. A modular detection pipeline that combines three complementary unsupervised models on the same equity LOB dataset (TOTF.PA, Euronext Paris): a Transformer bottleneck autoencoder inspired by Poutré et al. [4], coupled with a Nyström-approximated One-Class SVM in latent space; a Probabilistic Neural Network (PNN) that predicts skew-normal price move distributions following the framework of Fabre and Challet [1]; and a Probabilistic Robust Autoencoder (PRAE) with learnable per-sample gates [3]. Each model produces a scalar anomaly score per time step; their agreement identifies the highest-confidence anomalies.
2. A feature engineering library of approximately 80 features synthesising contributions from multiple sources: Hawkes-inspired memory features with spatial and temporal decay [1], weighted multi-level volume imbalance [7], event flow and cancellation rapidity [4], multi-level order flow imbalance, and standard microstructural indicators (spread, volatility, elasticity).
3. Application of the spoofing gain framework of Fabre and Challet [1] to equity LOB data. The PNN-predicted skew-normal parameters feed an expected cost model that quantifies the profitability of a hypothetical spoofing strategy at each time step, producing an interpretable economic measure of manipulation opportunity.

4. A systematic comparison of four threshold methods for converting continuous anomaly scores into binary alerts without labels: Peak Over Threshold (POT), Streaming POT (SPOT), Drift-aware Streaming POT (DSPOT) [5], and a rolling Benjamini-Hochberg False Discovery Rate controller (RFDR).
5. Interpretability analysis via Integrated Gradients [6] for the PNN and PRAE, and Grouped Occlusion for the Transformer+OC-SVM, identifying which feature families and which sides of the order book drive each model's anomaly scores.
6. Post-hoc characterisation of detected anomalies through HDBSCAN clustering of the three anomaly scores and microstructural jump analysis (bipower variation, variance ratio, CUSUM), providing an unsupervised taxonomy of anomaly types.
7. Cross-year robustness evaluation: models trained on 2015 data are tested on 2009, 2010, and 2017 data, and models trained on 2017 data are tested on the same periods, assessing whether the pipeline detects consistent patterns across different market regimes.

1.4 Report Outline

Chapter 2 reviews the literature on limit order book microstructure, market manipulation, and the detection methods that this project builds on. Chapter 3 formalises the feature engineering pipeline, the three detection models with their loss functions, the threshold methods, the spoofing gain framework, and the interpretability and post-hoc analysis tools. Chapter 4 describes the TOTF.PA dataset, the preprocessing steps, the sequential training protocol, and the experimental setup. Chapter 5 presents the experimental findings: model diagnostics, anomaly detection rates, threshold comparisons, sensitivity analysis, anomaly clustering, jump analysis, and cross-year robustness. Chapter 6 discusses the limitations of the approach, compares the three detection models, and interprets the detected anomalies in light of known manipulation signatures. Chapter 7 summarises the contributions and outlines directions for future work.

Chapter 2

Background and Related Work

This chapter provides the conceptual vocabulary and critical perspective needed to evaluate the methodological choices presented in Chapter 3. Each section introduces a class of approaches, explains what it is good at, where it struggles, and what assumptions it makes. Formal definitions, model architectures, and training procedures are deferred to Chapter 3.

2.1 Limit Order Books and Market Microstructure

A limit order book records all outstanding buy and sell orders for a security, organised by price and submission time. At time t, the book state consists of L price levels on each side. The bid (buy) side lists prices $p_1^b > p_2^b > \cdots > p_L^b$ with corresponding volumes $v_1^b, v_2^b, \ldots, v_L^b$. The ask (sell) side lists prices $p_1^a < p_2^a < \cdots < p_L^a$ with volumes $v_1^a, v_2^a, \ldots, v_L^a$. The best bid is p_1^b, the best ask is p_1^a, the mid-price is $p_{\text{mid}} = (p_1^a + p_1^b)/2$, and the spread is $s = p_1^a - p_1^b$.

Three event types modify the book state. A *limit order* adds volume at a specified price level, increasing liquidity. A *cancellation* removes a previously placed limit order, reducing liquidity at that level. A *market order* (equivalently, a trade) consumes volume at the best price on the opposite side, removing liquidity and potentially shifting the best price.

The instantaneous distribution of volume across price levels and the temporal sequence of these events jointly determine short-term price dynamics. Volume imbalance, the normalised excess of bid over ask volume at a given level, is a widely used summary statistic of the book state. Tao et al. [7] extend the single-level imbalance to a weighted multi-level measure that aggregates supply and demand across several price levels, and show on TMX (Toronto Stock Exchange) data that this quantity is directly linked to the probability distribution of subsequent price moves. Fabre and Challet [1] complement imbalance with Hawkes-inspired features that capture both the size and the spatial placement depth of new limit orders, demonstrating on cryptocurrency data that these features predict the conditional distribution of mid-price movements.

2.2 Market Manipulation Strategies

Spoofing consists of submitting large, visible limit orders with no intention of execution to create a false impression of supply or demand. The lifecycle of a spoofing event follows a characteristic sequence: (1) the manipulator places one or more large limit orders on one side of the book, shifting the perceived imbalance; (2) the mid-price moves in the direction favoured by the manipulator; (3) the manipulator executes a genuine order on the opposite side at the improved price; (4) the non-bona-fide orders are cancelled. Layering extends spoofing by

stacking multiple orders across several price levels on the same side, amplifying the perceived depth [1]. Quote stuffing is a related strategy in which a large number of orders are rapidly submitted and cancelled to slow down other participants' systems and exploit the resulting latency advantage.

Both spoofing and layering are prohibited under the European Market Abuse Regulation (EU MAR, Article 12) and the US Dodd-Frank Act (Section 747). Detection is challenging because the same order flow patterns can arise from legitimate trading strategies, such as aggressive liquidity provision or inventory management. Tao et al. [7] formalise the detection problem by modelling the imbalance shift caused by a spoof order and comparing the observed pre-trade imbalance distribution to its long-run counterpart via the Wasserstein distance. Fabre and Challet [1] take an economic approach: rather than asking whether an order flow pattern looks abnormal, they ask whether a rational spoofer could profit from the current book state, defining the spoofing gain as the difference in expected execution cost between a scenario where the spoof order is present and one where it is absent.

A binary classifier that labels each event as manipulative or not is poorly suited to this problem. No ground-truth labels exist at the order level; manipulation is a spectrum of intent rather than a discrete category; and a probabilistic score is more actionable than a binary flag because it allows ranking events by severity and calibrating alert thresholds to a desired false positive rate.

2.3 Anomaly Detection in Financial Data

In the absence of ground-truth labels for manipulation, detection must proceed unsupervised. A central difficulty shared by all unsupervised approaches is the *contamination problem*: training data collected from a real market inevitably contains some anomalous events, and their presence can degrade the learned model of normality. This section surveys three families of approaches and discusses their relative strengths and weaknesses for the LOB anomaly detection problem.

2.3.1 Reconstruction-Based Approaches

Autoencoders learn a compressed representation of the training data and reconstruct each input from that representation. When the training set is predominantly normal, the model learns to reconstruct normal patterns well; anomalous inputs produce high reconstruction error because they deviate from the learned structure. This family has several merits: it requires no labels, captures complex multivariate dependencies, and generalises across instruments and time periods without task-specific supervision.

For order book sequences, the choice of encoder architecture matters. Convolutional and recurrent architectures capture local patterns effectively but struggle with long-range dependencies between distant price levels or between events separated by many time steps. The Transformer architecture addresses this limitation through self-attention, which allows each position in the sequence to attend directly to every other position. Multi-head attention enables the model to simultaneously capture different aspects of the book state, such as volume dynamics at the top of the book, spread behaviour, and order arrival patterns deeper in the queue. Poutré et al. [4] demonstrate on NASDAQ LOBSTER data that a Transformer autoencoder combined with a one-class classifier in the latent space outperforms the reconstruction error alone for detecting synthetically injected manipulation patterns (quote stuffing, layering, and pump-and-dump).

A limitation of deterministic autoencoders is that the reconstruction error is a noisy proxy for anomaly severity: it does not distinguish between inputs that are merely unusual and inputs

that fall outside the training distribution entirely. A probabilistic formulation, in which the autoencoder outputs a distribution over reconstructions rather than a point estimate, addresses this limitation by providing an uncertainty signal. Lindenbaum et al. [3] go further: rather than modifying the reconstruction itself, they introduce learnable per-sample gates that allow the training procedure to downweight suspected anomalies, so that the autoencoder focuses on reconstructing clean data even when the training set is contaminated. This makes the model robust to the contamination problem without requiring an explicit data-cleaning step.

2.3.2 Probabilistic Scoring via Synthetic Manipulation

An alternative to asking "does this look abnormal?" is to ask "how much could a rational spoofer gain from this book state?" Fabre and Challet [1] pursue this reframing by training a probabilistic neural network that predicts the full conditional distribution of the next mid-price move. The predicted distribution serves two purposes: its negative log-likelihood provides a general anomaly score (events where the realised price move is unlikely under the model receive high scores), and its parameters feed an expected cost model that quantifies the profitability of a hypothetical spoofing strategy at each time step.

The merits of this approach are substantial. The spoofing gain score has a direct economic interpretation, which makes it more actionable for a market surveillance system than a generic reconstruction error. Training on the predictive task sidesteps the need for labelled manipulation data: the model learns normal price dynamics, and deviations from that model are flagged regardless of their cause. The probabilistic output naturally quantifies uncertainty over the predicted price move, which matters for heavy-tailed financial data where point predictions are unreliable.

The limitations are equally clear. The spoofing gain depends on a model of the spoofer's strategy (order size, placement distance, fee structure), so it may miss manipulation patterns that deviate from that model. The approach was developed and evaluated on cryptocurrency LOB data (BTC-USD and ETH-USD on Coinbase) [1], where the absence of anti-manipulation regulation results in a higher base rate of manipulative behaviour; its effectiveness on regulated equity markets, where manipulation is rarer and subtler, has not been established in the literature.

2.3.3 One-Class and Density-Based Approaches

One-class classification methods learn a tight boundary around the training data in a transformed feature space and flag anything outside that boundary as anomalous. The One-Class SVM (OC-SVM) is the most widely used instance: it finds a hyperplane in a kernel-induced feature space that separates the data from the origin with maximum margin, controlled by a parameter ν that bounds the fraction of training points allowed outside the boundary.

The strengths of the OC-SVM are computational efficiency and theoretical simplicity. It does not require specifying what an anomaly looks like, only what normal data looks like, and the kernel trick allows it to capture non-linear structure in the input space. However, it does not scale well to high-dimensional feature spaces without careful kernel design: in the approximately 80-dimensional feature space typical of LOB anomaly detection, the RBF kernel becomes poorly calibrated and the decision boundary may either collapse or become too loose. Low-rank kernel approximations (such as the Nyström method) partially address this issue by projecting the kernel evaluation into a tractable subspace, enabling GPU-accelerated training. A further limitation is that the OC-SVM does not produce a calibrated probability; its decision function is a signed distance, not a likelihood, which complicates downstream interpretation and threshold calibration.

For the LOB problem, the OC-SVM is most effective when applied not to raw features but to a learned latent representation. Poutré et al. [4] demonstrate this by fitting an OC-SVM in the bottleneck space of a Transformer autoencoder, where the reduced dimensionality and the learned structure of the representation make the kernel boundary more discriminative. This two-stage approach combines the representation power of reconstruction-based methods with the boundary-learning capacity of one-class classification.

2.4 Thresholding Methods for Anomaly Detection

Any continuous anomaly score requires a threshold to produce a detection decision. The naive approach, a fixed percentile of the score distribution, assumes that the distribution is stationary and well-behaved. In financial data, score distributions are heavy-tailed and non-stationary: a fixed percentile will produce too many false positives during volatile periods and miss anomalies during quiet periods. Setting thresholds without labels is therefore a problem in its own right.

Extreme Value Theory (EVT). EVT models the tail of the score distribution rather than the bulk, fitting a Generalised Pareto Distribution to the exceedances above a high initial threshold. This is well-suited to heavy-tailed data because the statistical guarantee applies specifically to extreme quantiles, where fixed-percentile methods are least reliable. Siffer et al. [5] extend the batch EVT estimator to streaming settings with two algorithms: SPOT, which updates the tail model as new peaks arrive, and DSPOT, which adds a drift correction that tracks shifts in the baseline score level over time. The main assumption is that the tail of the score distribution is well-approximated by the GPD, which must be verified empirically for each model output.

False Discovery Rate (FDR) Control. FDR-based procedures, such as the Benjamini-Hochberg method, control the expected proportion of false positives among all flagged events. This is appropriate when anomalies are rare and the cost of false positives is high, because it bounds the discovery error rate rather than the absolute false positive count. The requirement is a set of p-values or a reference null distribution for the anomaly score, which must be estimated from the data when no parametric form is available.

Rolling FDR (RFDR). RFDR adapts the Benjamini-Hochberg procedure to a streaming setting by maintaining a sliding window of recent anomaly scores. Within each window, scores are log-transformed, converted to robust z-scores using the median absolute deviation, and mapped to p-values under a Gaussian assumption; the BH procedure then determines a dynamic threshold that controls the false discovery rate at level α. Because the null distribution is re-estimated at every step from the local window, RFDR tracks non-stationarity in the score distribution without requiring an explicit drift model.

Implicit Model Thresholds. Some detection methods produce a threshold as a byproduct of training rather than as a post-hoc step. The OC-SVM decision boundary, for instance, defines an implicit threshold in the feature space: points with negative decision function values are classified as anomalous. This is both a strength (the threshold is jointly optimised with the model) and a weakness (the threshold is not interpretable independently of the model and cannot be recalibrated without retraining).

2.5 Feature Attribution Methods

A detection score alone is not actionable for a market surveillance analyst: the analyst needs to know which features of the order book drove the score and whether those features align with known manipulation signatures. Attribution for sequential financial data is harder than for static inputs because features at different time steps and price levels are structurally correlated,

and because the underlying distribution shifts over the trading day.

Two broad families of attribution methods are relevant. Gradient-based methods, such as Integrated Gradients [6], accumulate the model's gradient along a path from a baseline input to the actual input, producing per-feature importance scores that are theoretically principled (they satisfy completeness and sensitivity axioms) and computationally tractable. Their limitation is that they require a differentiable end-to-end model, which excludes pipelines where a non-differentiable component (such as an OC-SVM) sits between the learned representation and the final score.

For non-differentiable pipelines, ablation-based methods provide an alternative. Grouped occlusion replaces semantically related groups of features (all bid-side features, all Hawkes features, all features at a given price level) with a baseline value and measures the resulting change in the anomaly score. This approach is model-agnostic and handles the structured correlation among LOB features by operating at the group level rather than the individual feature level. The trade-off is lower resolution: it identifies which feature families matter, not which individual features within a family are most important.

Chapter 3

Methodology

3.1 Feature Engineering

The raw limit order book data for TOTF.PA (TotalEnergies, Euronext Paris) records the state of the book at each update event during continuous trading hours (09:00–17:30 CET). Each row contains an Excel serial timestamp t (column `xltime`, where the fractional part encodes the time of day) and the price and volume at $L = 10$ levels on each side: $p_1^b, V_1^b, \ldots, p_{10}^b, V_{10}^b$ for the bid and $p_1^a, V_1^a, \ldots, p_{10}^a, V_{10}^a$ for the ask, giving 41 columns per row. The data is event-driven: a new row is recorded whenever a limit order, cancellation, or trade modifies the visible book state, so the inter-event spacing is irregular.

Feeding the 40 raw price-volume columns directly to a model is wasteful and fragile. Prices at adjacent levels are highly correlated, volumes are non-stationary, and the absolute price level carries no information about short-term dynamics. Engineered features reduce the input dimensionality to $d \approx 80$ informative quantities that are more stationary, more interpretable, and encode the domain knowledge needed to distinguish normal from anomalous order flow.

The features are organised into seven groups: (1) order book imbalance, capturing supply-demand asymmetry across depth levels; (2) price dynamics and microstructure, summarising price movements and their relationship to volume; (3) liquidity and elasticity, measuring the cost and shape of the book; (4) volatility proxies; (5) event flow and rapidity, characterising the intensity and type of order arrivals; (6) Hawkes-inspired memory features, capturing self-exciting patterns in order flow with spatial and temporal decay; and (7) order flow imbalance, measuring signed directional pressure from volume changes conditioned on price movements. Preprocessing and scaling are described in Section 3.1.8.

3.1.1 Order Book Imbalance

Imbalance features quantify the asymmetry between supply and demand at various depths of the book. Spoofing and layering distort this asymmetry by injecting volume on one side; imbalance measures are therefore among the most direct indicators of manipulative intent.

The Level-1 imbalance compares bid and ask volume at the best quotes:

$$I_t^{(1)} = \frac{V_{1,t}^b - V_{1,t}^a}{V_{1,t}^b + V_{1,t}^a}. \tag{3.1}$$

Values near $+1$ indicate strong buy-side pressure; values near -1 indicate sell-side pressure.

The Level-5 imbalance aggregates volume across the top five levels on each side:

$$I_t^{(5)} = \frac{\sum_{i=1}^{5} V_{i,t}^b - \sum_{i=1}^{5} V_{i,t}^a}{\sum_{i=1}^{5} V_{i,t}^b + \sum_{i=1}^{5} V_{i,t}^a}. \tag{3.2}$$

A divergence between $I_t^{(1)}$ and $I_t^{(5)}$ suggests that volume is concentrated away from the best quotes, a pattern consistent with layering.

To weight deeper levels more flexibly, we follow Tao et al. [7] and define a weighted multi-level imbalance over $L = 5$ levels with weight vector $\mathbf{w} = (w_1, \ldots, w_5)$:

$$I_t^{(\mathbf{w})} = \frac{\sum_{i=1}^{5} w_i V_{i,t}^b}{\sum_{i=1}^{5} w_i V_{i,t}^b + \sum_{i=1}^{5} w_i V_{i,t}^a}. \tag{3.3}$$

This quantity lies in $[0, 1]$; values above 0.5 indicate net bid-side weight. Three weight configurations are used: *decreasing* $\mathbf{w} = (0.1, 0.1, 0.2, 0.2, 0.4)$, which emphasises deeper levels where spoof orders are less likely to be executed; *increasing* $\mathbf{w} = (0.4, 0.2, 0.2, 0.1, 0.1)$, which emphasises the best quotes; and *constant* $\mathbf{w} = (0.2, 0.2, 0.2, 0.2, 0.2)$, which weights all five levels equally.

3.1.2 Price Dynamics and Microstructure

This group captures the immediate state of the price process and its first- and second-order dynamics.

The mid-price and bid-ask spread are:

$$p_{\text{mid},t} = \frac{p_{1,t}^a + p_{1,t}^b}{2}, \qquad s_t = p_{1,t}^a - p_{1,t}^b. \tag{3.4}$$

The micro-price deviation measures how far the L1-imbalance-adjusted fair value lies from the mid-price:

$$\delta_t^{\text{micro}} = I_t^{(1)} \cdot \frac{s_t}{2}. \tag{3.5}$$

A large deviation suggests that the balance of volume at the best quotes is pulling the fair value away from the geometric centre of the spread.

The volume depth ratio measures the concentration of volume away from the best quote on each side $s \in \{b, a\}$:

$$D_t^s = \frac{\sum_{i=2}^{5} V_{i,t}^s}{V_{1,t}^s}. \tag{3.6}$$

High values indicate that most liquidity sits behind the best quote, a signature of layering where large volumes are placed at deeper levels.

Log returns, velocity, and acceleration of the mid-price are:

$$r_t = \ln\left(\frac{p_{\text{mid},t}}{p_{\text{mid},t-1}}\right), \quad v_t = p_{\text{mid},t} - p_{\text{mid},t-1}, \quad a_t = v_t - v_{t-1}. \tag{3.7}$$

The net order flow at Level 1 isolates volume changes that occur at a fixed price, filtering out changes caused by price-level shifts:

$$F_t^s = \Delta V_{1,t}^s \cdot \mathbf{1}\left[p_{1,t}^s = p_{1,t-1}^s\right], \qquad s \in \{b, a\}, \tag{3.8}$$

where $\Delta V^s_{1,t} = V^s_{1,t} - V^s_{1,t-1}$. A large positive F^b_t followed by a rapid cancellation is a textbook spoofing pattern.

Volume deltas $\Delta V^b_{1,t}$ and $\Delta V^a_{1,t}$ at the best quotes are also retained as raw features, along with the rolling standard deviation of the mid-price over a window of $W = 50$ events.

3.1.3 Liquidity and Elasticity

These features describe how much it costs to execute a given order size and how steeply the book's price schedule rises with volume.

The sweep-to-fill cost is the volume-weighted average price paid to execute a target order of size $Q = 1{,}000$ units by walking through $L = 10$ levels. The value $Q = 1{,}000$ was chosen empirically to represent a moderately sized institutional order for TOTF.PA; no reference paper prescribes a specific target size, and a sensitivity analysis over Q is deferred to Chapter 4. Denoting by $C^s_t(Q)$ the resulting average execution price on side s, the reported features are the excess costs relative to the mid-price:

$$\mathrm{SC}^a_t = C^a_t(Q) - p_{\mathrm{mid},t}, \qquad \mathrm{SC}^b_t = p_{\mathrm{mid},t} - C^b_t(Q). \tag{3.9}$$

Both quantities are non-negative under normal conditions. Large values indicate thin liquidity: a spoofing episode that removes genuine orders from the book can cause a spike in sweep cost.

The book slope (elasticity) measures the sensitivity of price to cumulative volume. For each side s, a linear regression of price on log cumulative volume across the first $D = 5$ levels gives (Tao et al. [7] calibrate a comparable depth parameter to values between 3 and 5 depending on the stock; Fabre and Challet [1] use $K = 5$ as a representative depth): The depth $D = 5$ is consistent with the range used by Tao et al. [7] (who calibrate 3 to 5 depending on the stock) and Fabre and Challet [1] (who use $K = 5$); an ablation over D is reported in Chapter 4.

$$p^s_{i,t} \approx \alpha^s_t + \beta^s_t \ln\left(\sum_{j=1}^{i} V^s_{j,t} + 1\right), \qquad i = 1, \ldots, D, \tag{3.10}$$

and the feature is the absolute slope $|\beta^s_t|$. A steep slope (large $|\beta^s_t|$) indicates low liquidity, where a small volume would cause a large price change; a flat slope indicates a deep, resilient book.

3.1.4 Volatility

Local volatility features capture the recent variability of the price process and of event timing.

Rolling standard deviations of log returns are computed over two horizons:

$$\sigma^{(W)}_t = \sqrt{\frac{1}{W-1}\sum_{k=0}^{W-1}(r_{t-k} - \bar{r}_t)^2}, \qquad W \in \{50, 100\}. \tag{3.11}$$

The normalised price range over a window of $W = 50$ events measures extreme movements:

$$R_t = \frac{\max_{k\in[t-W+1,t]} p_{\mathrm{mid},k} - \min_{k\in[t-W+1,t]} p_{\mathrm{mid},k}}{p_{\mathrm{mid},t}}. \tag{3.12}$$

A sudden spike in R_t without a corresponding increase in $\sigma^{(50)}_t$ may indicate a transient price dislocation caused by manipulation.

The absolute velocity $|r_t|$ captures the instantaneous magnitude of price movement without directional information.

The inter-event time $\Delta t_t = t - t_{t-1}$ (in fractions of a day, following the Excel `xltime` convention) and its logarithm $\ln(\Delta t_t + \epsilon)$, with $\epsilon = 10^{-6}$, are included to provide the models with information about event timing. Abnormally short inter-event times can signal quote-stuffing activity.

3.1.5 Event Flow and Rapidity

Following Poutré et al. [4], each LOB update is classified as either a cancellation or a trade based on observable price and volume changes. A cancellation on side s is identified when the best price is unchanged and volume decreases ($p^s_{1,t} = p^s_{1,t-1}$ and $\Delta V^s_{1,t} < 0$). A trade on the bid (ask) side is identified when the opposite ask (bid) price changes, indicating that a market order has consumed liquidity.

The event size features record the magnitude of each event type:

$$\text{Size}_t^{\text{cancel},s} = |\Delta V^s_{1,t}| \cdot \mathbf{1}[\text{cancel}^s_t], \qquad \text{Size}_t^{\text{trade},s} = V^s_{1,t-1} \cdot \mathbf{1}[\text{trade}^s_t]. \tag{3.13}$$

Each of these four quantities, along with the raw Level-1 volume, is smoothed with a simple moving average over $W_{\text{SMA}} = 10$ events, following the feature definitions in Poutré et al. [4]. The window size $W_{\text{SMA}} = 10$ follows the implementation of Poutré et al. [4], who adopt this value without ablation; we retain it unchanged and note that no sensitivity analysis over W_{SMA} is performed in this work.

Rapidity features measure the density of events per unit of calendar time:

$$\rho_t^{\text{type},s} = \frac{\mathbf{1}[\text{type}^s_t]}{\Delta t_t}, \qquad \text{type} \in \{\text{cancel}, \text{trade}\}, \quad s \in \{b, a\}. \tag{3.14}$$

High cancellation rapidity on one side of the book is a signature of quote stuffing, where a manipulator floods the exchange with rapid submissions and cancellations.

Bid- and ask-side log returns ($\ln(p^s_{1,t}/p^s_{1,t-1})$ for $s \in \{b, a\}$) and price speed (log return divided by Δt_t) complete this group.

3.1.6 Hawkes-Inspired Memory Features

Following Fabre and Challet [1], we define order flow variables inspired by the self-exciting structure of Hawkes processes. Let $(N_t^{L,b})_{t\geq 0}$ and $(N_t^{L,a})_{t\geq 0}$ denote the point processes counting the arrivals of buy and sell limit orders, and let $(N_t^{M,b})_{t\geq 0}$ and $(N_t^{M,a})_{t\geq 0}$ denote the point processes of sell and buy marketable orders (i.e. orders that cause transactions on the bid and ask sides respectively). Each limit order carries two marks: a volume $v_t^{L,s} > 0$ and a distance of placement $\delta_t^{L,s} > 0$ from the best price, for $s \in \{b, a\}$. Each marketable order carries a volume mark $v_t^{M,s} > 0$.

The limit-order and marketable-order flow variables are defined as

$$L_t^s(\phi) = \int_{(-\infty,t]} \phi(t-u,\, v_u^{L,s},\, \delta_u^{L,s})\, dN_u^{L,s}, \qquad s \in \{b, a\}, \tag{3.15}$$

$$M_t^s(\psi) = \int_{(-\infty,t]} \psi(t-u,\, v_u^{M,s})\, dN_u^{M,s}, \qquad s \in \{b, a\}, \tag{3.16}$$

where ϕ is a positive kernel that weights each past limit order by its elapsed time, volume, and distance of placement, and ψ is a positive kernel that weights each past marketable order by its elapsed time and volume.

The kernels take exponential form with a linear volume function $f(v) = v$:

$$\phi_{\beta,\eta}(t, v, \delta) = e^{-\beta t}\, v\, e^{-\eta\delta}, \qquad \psi_\beta(t, v) = e^{-\beta t}\, v, \tag{3.17}$$

where $\beta > 0$ controls the temporal decay rate and $\eta \geq 0$ controls the spatial decay rate. The exponential time kernel is chosen for two reasons: it satisfies the Markov property, which permits efficient recursive updates in a streaming setting, and combining multiple time scales β approximates the slowly decaying empirical memory of order flow across several decades [1].

Two families of kernels are constructed by varying β over a finite set $\boldsymbol{\beta}$ and η over a finite set $\boldsymbol{\eta}$:

$$\boldsymbol{\phi} = \{\phi_{\beta,\eta} : \beta \in \boldsymbol{\beta},\, \eta \in \boldsymbol{\eta}\}, \qquad \boldsymbol{\psi} = \{\psi_\beta : \beta \in \boldsymbol{\beta}\}. \tag{3.18}$$

The corresponding feature sets at time t are

$$\mathcal{L}_t = \big\{(L_t^b(\phi),\, L_t^a(\phi)) : \phi \in \boldsymbol{\phi}\big\}, \qquad \mathcal{M}_t = \big\{(M_t^b(\psi),\, M_t^a(\psi)) : \psi \in \boldsymbol{\psi}\big\}. \tag{3.19}$$

Each element of $\mathcal{L}_t$ captures limit-order flow at a specific combination of time and distance scales; each element of $\mathcal{M}_t$ captures marketable-order flow at a specific time scale. Small η values give nearly equal weight to orders at all depths; large η values concentrate attention on orders near the best quotes, so that distant spoof orders receive negligible weight.

Adaptation to Level-2 data. Fabre and Challet [1] work with Level-3 data, where each order event is individually observed with its volume and distance of placement. Our dataset provides Level-2 snapshots (aggregate volume at each of $L = 10$ price levels), so the integrals in Equations (3.15)-(3.16) are approximated as follows. At each event t, the limit-order volume at level i on side s is inferred from positive volume changes at unchanged price levels: $[\Delta V_{i,t}^s]^+ \cdot \mathbf{1}[p_{i,t}^s = p_{i,t-1}^s]$. The distance mark $\delta_{i,t}^{L,s}$ is set to $c \cdot |p_{i,t}^s - p_{\mathrm{mid},t}|$, where $c = 10^4$ converts Euro-denominated distances to a dimensionless quantity comparable to basis points. The scaling constant $c = 10^4$ was chosen empirically so that the typical distance of a Level-1 order (~ 0.01 EUR for TOTF.PA) maps to a dimensionless value of order 10^2, placing it in a comparable range to the basis-point distances used by Fabre and Challet [1] on cryptocurrency data; c is dataset-specific and should be recalibrated for instruments with a different tick size. Marketable-order flow is inferred from volume decreases at changing price levels. The continuous-time exponential decay $e^{-\beta t}$ is replaced by a discrete EWMA with half-life parameter β (measured in events), applied to each side and kernel configuration independently.

Parameter grid. Following Fabre and Challet [1], the time scales are $\boldsymbol{\beta} = \{10, 100, 1{,}000\}$ and the distance scales are $\boldsymbol{\eta} = \{0.001, 0.1, 1.0, 10.0\}$ (expressed in bp^{-1} in the original paper). These grid values follow Fabre and Challet [1], who select them on the basis of a multi-scale coverage argument without a formal ablation study; we adopt the same grid and do not perform an independent ablation in this work. With two sides of the book, $|\boldsymbol{\beta}| \times |\boldsymbol{\eta}| \times 2 = 24$ limit-order flow features form $\mathcal{L}_t$, and $|\boldsymbol{\beta}| \times 2 = 6$ marketable-order flow features form $\mathcal{M}_t$.

Short- and long-horizon flows. The half-lives $h_s = 5$ and $h_\ell = 50$ were chosen empirically to capture short-term (a few events) and medium-term (tens of events) order flow memory at the top of the book; no reference paper prescribes specific values, and a sensitivity analysis is deferred to Chapter 4. In addition to the kernel-weighted features above, limit-order and marketable-order flows at the best level (Level 1) are smoothed at two fixed half-lives ($h_s = 5$ and $h_\ell = 50$ events) for the limit component and at $h_s = 5$ for the market component, yielding six features: $L_t^b(h_s)$, $L_t^a(h_s)$, $M_t^b(h_s)$, $M_t^a(h_s)$, $L_t^b(h_\ell)$, $L_t^a(h_\ell)$. These correspond to the special case $\eta = 0$ (no spatial decay) with a single time scale, and capture the short- and long-term memory of order flow at the top of the book.

Deep order insertions. Volume increases at Level 5 are tracked separately to capture layering activity deep in the book. The positive volume change at Level 5, smoothed with an EWMA of

half-life $h_d = 10$, produces two additional features (bid and ask). The half-life $h_d = 10$ and the choice of Level 5 as the monitoring depth were set empirically: Level 5 lies deep enough that large insertions there are unlikely to be filled, consistent with the layering signature described by Fabre and Challet [1], and $h_d = 10$ provides smoothing over a short horizon to reduce noise; sensitivity to both choices is deferred to Chapter 4.

3.1.7 Order Flow Imbalance

Order flow imbalance (OFI) measures the net signed change in supply and demand at each price level, conditioned on whether the price at that level has moved. Unlike the imbalance features of Section 3.1.1, which compare static snapshots of volume, OFI captures the directional pressure exerted by incoming orders.

At each level $i \in \{1, \ldots, 5\}$, the bid-side flow is:

$$\phi^b_{i,t} = \begin{cases} V^b_{i,t} & \text{if } p^b_{i,t} > p^b_{i,t-1}, \\ V^b_{i,t} - V^b_{i,t-1} & \text{if } p^b_{i,t} = p^b_{i,t-1}, \\ -V^b_{i,t-1} & \text{if } p^b_{i,t} < p^b_{i,t-1}, \end{cases} \tag{3.20}$$

and the ask-side flow $\phi^a_{i,t}$ follows the symmetric convention (positive when the ask price improves, i.e. decreases). The OFI at level i is

$$\text{OFI}_{i,t} = \phi^b_{i,t} - \phi^a_{i,t}. \tag{3.21}$$

Positive values indicate net buying pressure; negative values indicate net selling pressure. Computing OFI at five separate levels allows the models to distinguish pressure at the best quotes from pressure deeper in the book, which is relevant for detecting layering.

3.1.8 Preprocessing and Scaling

Before the feature matrix enters any model, four cleaning steps are applied in sequence. The value $W_{\text{warmup}} = 3{,}000$ was chosen empirically to ensure that all rolling windows (up to $W = 100$) and EWMA features (with half-life up to $h_\ell = 50$) are fully populated before the output is consumed by any model; a sensitivity analysis is reported in Chapter 4. First, the initial $W_{\text{warmup}} = 3{,}000$ rows of each trading day are discarded to allow all EWMA and rolling-window features to stabilise. Second, infinite values (arising from division by zero in rare edge cases) are replaced by zero. Third, each feature column is clipped at its 0.1% and 99.9% quantiles to remove extreme outliers while preserving the tail structure needed for anomaly detection. Fourth, any column with standard deviation below 10^{-9} is dropped as uninformative.

Three scaling methods are available, selected per experiment. *MinMax* scaling maps each feature to $[0, 1]$. *Standard* scaling centres each feature to zero mean and unit variance. The default method, following Fabre and Challet [1], is an empirical Box-Cox transformation. For each feature j, the Box-Cox transform with parameter $\lambda_j > 0$ is

$$T_{\lambda_j}(x^{(j)}) = \frac{(x^{(j)})^{\lambda_j} - 1}{\lambda_j}, \tag{3.22}$$

with the convention that $T_0(x) = \ln(x)$. The parameter λ_j is chosen per feature to maximise the Gaussian log-likelihood of the transformed values on the training set. Before transformation, features with non-positive values are shifted by $|\min(x^{(j)})| + \epsilon$ (with $\epsilon = 10^{-6}$) to ensure strictly positive inputs; the shift is stored and applied consistently at inference time. After

transformation, each feature is standardised to zero mean and unit variance using the training-set statistics:

$$z_{\lambda_j}(x^{(j)}) = \frac{T_{\lambda_j}(x^{(j)}) - \bar{T}_{\lambda_j}}{\sqrt{S_{\lambda_j}}}, \tag{3.23}$$

where $\bar{T}_{\lambda_j}$ and S_{λ_j} are the empirical mean and variance of the transformed feature on the training set. This two-stage procedure addresses the heavy tails typical of LOB features, which can cause unstable weight updates during training. A legacy *Quantile* scaler (quantile transformation to a Gaussian followed by standard scaling) is retained for ablation purposes. In all cases, the scaler is fit on the training partition only and then applied to validation and test data, preventing information leakage across splits.

3.2 Detection Models

Three complementary unsupervised models are trained on the engineered feature set described in Section 3.1. Each model embodies a different inductive bias: the Transformer bottleneck autoencoder with One-Class SVM learns a compressed latent representation and fits a one-class boundary in that space; the probabilistic neural network predicts the full conditional distribution of the next mid-price move, providing both a likelihood-based anomaly score and an economic spoofing gain estimate; the probabilistic robust autoencoder learns to reconstruct normal patterns while explicitly down-weighting suspected anomalies during training. The diversity of approaches is intentional: reconstruction-based, density-estimation-based, and robust methods surface different signatures of anomalous order flow, and their agreement identifies the highest-confidence detections.

3.2.1 Transformer Bottleneck Autoencoder with One-Class SVM

This detector follows the two-stage framework of Poutré et al. [4]: a Transformer bottleneck autoencoder is trained to reconstruct LOB feature sequences, then its encoder is frozen and a One-Class SVM is fitted on the bottleneck representations. The anomaly score is the negated OC-SVM decision function, so that higher values indicate greater anomalousness.

Encoder. The input to the encoder is a subsequence $\mathbf{S}_t \in \mathbb{R}^{k \times d}$, where k is the sequence length and d the number of features. A linear embedding projects each feature vector from $\mathbb{R}^d$ to $\mathbb{R}^{d_{\text{model}}}$. The embedded sequence is scaled by $\sqrt{d_{\text{model}}}$ and summed with sinusoidal positional encodings following Vaswani et al. [8], providing the model with information about relative position within the subsequence. The encoded sequence passes through N_{enc} Transformer encoder layers, each consisting of multi-head self-attention with h heads followed by a position-wise feedforward network. After the final encoder layer, the output matrix $\mathbf{E}_t \in \mathbb{R}^{k \times d_{\text{model}}}$ is flattened to a vector $\mathbf{e}_t \in \mathbb{R}^{k \cdot d_{\text{model}}}$ and projected through a linear bottleneck to produce the representation:

$$\mathbf{z}_t = \mathbf{W}_b \, \mathbf{e}_t + \mathbf{b}_b \in \mathbb{R}^{d_{\text{rep}}}, \tag{3.24}$$

where $\mathbf{W}_b \in \mathbb{R}^{d_{\text{rep}} \times k \cdot d_{\text{model}}}$. Because $d_{\text{rep}} \ll k \cdot d$, the bottleneck forces the encoder to discard redundant information and retain only the features most useful for reconstruction. Anomalous subsequences, which deviate from the patterns seen during training, are expected to occupy regions of the latent space distinct from normal data.

Decoder. The decoder mirrors the encoder in reverse. A linear layer expands the bottleneck vector $\mathbf{z}_t$ back to $\mathbb{R}^{k \cdot d_{\text{model}}}$, which is reshaped to $\mathbb{R}^{k \times d_{\text{model}}}$. This matrix passes through N_{dec} Transformer layers (using the same `TransformerEncoderLayer` architecture as the encoder, without cross-attention or causal masking) and a final linear projection from $\mathbb{R}^{d_{\text{model}}}$ to $\mathbb{R}^d$ pro-

duces the reconstruction $\hat{\mathbf{S}}_t$. Both the encoder and decoder use the same number of layers ($N_{\text{enc}} = N_{\text{dec}} = N$) and the same model dimension d_{model}, making the architecture symmetric.

Training objective. The autoencoder is trained to minimise the mean squared error between the input and its reconstruction:

$$\mathcal{L}_{\text{AE}}(\Theta) = \frac{1}{|\mathcal{D}|} \sum_{\mathbf{S}_t \in \mathcal{D}} \left\| \mathbf{S}_t - \hat{\mathbf{S}}_t \right\|_F^2, \tag{3.25}$$

where $\| \cdot \|_F$ denotes the Frobenius norm and $\mathcal{D}$ is the training set. The Adam optimiser [2] is used with a learning rate of $\eta = 10^{-4}$. Training proceeds for up to $E = 1{,}000$ epochs per day with early stopping on the validation loss (patience $P = 20$ epochs). Gradient norms are clipped to 1.0 to prevent instability.

Divergences from Poutré et al. The architecture parameters differ from those of Poutré et al. [4], who use $h = 8$ attention heads, a feedforward dimension $d_{\text{ff}} = 512$, and $N = 6$ Transformer layers, following Vaswani et al. [8]. Our implementation uses $h = 4$, $d_{\text{model}} = 64$ (with the default feedforward dimension $d_{\text{ff}} = 2{,}048$ from PyTorch's `TransformerEncoderLayer`), and $N = 2$, resulting in a substantially smaller model. This reduction was necessary because the TOTF.PA dataset is smaller than the five-stock LOBSTER dataset used in the original paper, and the larger architecture overfitted in preliminary experiments. The representation dimension ($d_{\text{rep}} = 128$) and the sequence length ($k = 25$) match the paper.

One-Class SVM on bottleneck representations. After the autoencoder is trained, the encoder is frozen and used to map every training subsequence to its bottleneck vector $\mathbf{z}_t \in \mathbb{R}^{d_{\text{rep}}}$. A One-Class SVM with a radial basis function (RBF) kernel is then fitted on these representations. The OC-SVM finds a hyperplane in the kernel-induced feature space $\mathcal{F}$ that separates the mapped data from the origin with maximum margin, solving:

$$\min_{\mathbf{w} \in \mathcal{F}, \xi, \rho} \quad \frac{1}{2}\|\mathbf{w}\|^2 - \rho + \frac{1}{\nu n} \sum_{i=1}^{n} \xi_i, \qquad \text{s.t.} \quad \mathbf{w}^\top \Phi(\mathbf{z}_i) \geq \rho - \xi_i, \quad \xi_i \geq 0, \tag{3.26}$$

where Φ is the feature map induced by the RBF kernel $k_\gamma(\mathbf{z}, \mathbf{z}') = \exp\!\left(-\gamma \|\mathbf{z} - \mathbf{z}'\|^2\right)$, $\nu \in (0, 1]$ bounds the fraction of training points allowed outside the boundary, and ξ_i are slack variables.

The decision function for a test point $\mathbf{z}$ is:

$$f(\mathbf{z}) = \mathbf{w}^\top \Phi(\mathbf{z}) - \rho. \tag{3.27}$$

Positive values indicate inliers; negative values indicate outliers. The anomaly score used downstream is $-f(\mathbf{z})$, so that higher scores correspond to greater anomalousness.

The OC-SVM is applied to the bottleneck rather than to the raw features or to the reconstruction error for two reasons. First, the bottleneck is a lower-dimensional ($d_{\text{rep}} = 128$ vs. $d \approx 80$ features $\times\, k = 25$ time steps $= 2{,}000$ raw dimensions), semantically compressed space where the OC-SVM can learn a tighter boundary. Second, Poutré et al. [4] show that the dissimilarity approach (OC-SVM on representations) substantially outperforms the reconstruction error alone, because normal data is more easily separable from anomalous data in the learned representation space than in the reconstruction error distribution.

The bandwidth parameter γ is set via the median heuristic:

$$\gamma = \frac{1}{\text{median}\left(\|\mathbf{z}_i - \mathbf{z}_j\|^2\right)}, \tag{3.28}$$

computed over a random subsample of 2,000 pairs from the training representations. This differs from Poutré et al. [4], who use $\gamma = 10/(d_{\text{rep}} \cdot \text{Var}(\mathbf{z}))$; the median heuristic was found

to produce better-calibrated kernel values in preliminary experiments on TOTF.PA data (the original formula yielded γ values large enough to collapse k_γ to near-zero between distinct points).

Nyström Approximation and SGD Training

The Nyström kernel approximation is a computational adaptation introduced in this project; it does not appear in Poutré et al. [4], who use the standard quadratic-programming solver from scikit-learn. The adaptation is motivated by scalability: a standard kernel SVM requires computing and storing an $n \times n$ kernel matrix, which is infeasible when n exceeds 10^5 training subsequences. The Nyström method approximates the kernel matrix using a random subset of $m \ll n$ landmark points, reducing memory from $O(n^2)$ to $O(nm)$ and enabling GPU-accelerated training.

The approximation proceeds as follows. A set of m landmark points $\mathbf{C} = \{\mathbf{c}_1, \ldots, \mathbf{c}_m\}$ is drawn uniformly at random from the training representations. The $m \times m$ kernel matrix $\mathbf{K}_{\mathrm{CC}} = [k_\gamma(\mathbf{c}_i, \mathbf{c}_j)]_{i,j}$ is computed and regularised by adding $10^{-6}\mathbf{I}_m$. Its eigendecomposition $\mathbf{K}_{\mathrm{CC}} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top$ yields the normalisation matrix $\mathbf{U}\boldsymbol{\Lambda}^{-1/2}$. The approximate feature map for any point $\mathbf{z}$ is:

$$\tilde{\Phi}(\mathbf{z}) = \mathbf{k}(\mathbf{z}, \mathbf{C}) \cdot \mathbf{U}\boldsymbol{\Lambda}^{-1/2} \in \mathbb{R}^m, \tag{3.29}$$

where $\mathbf{k}(\mathbf{z}, \mathbf{C}) = \big[k_\gamma(\mathbf{z}, \mathbf{c}_1), \ldots, k_\gamma(\mathbf{z}, \mathbf{c}_m)\big]$.

After applying this feature map, the OC-SVM reduces to a linear problem in $\mathbb{R}^m$. The weight vector $\mathbf{w} \in \mathbb{R}^m$ and the offset ρ are trained via stochastic gradient descent on the hinge-based objective:

$$\mathcal{L}_{\text{OC-SVM}}(\mathbf{w}, \rho) = \frac{1}{2}\|\mathbf{w}\|^2 - \rho + \frac{1}{\nu|\mathcal{B}|}\sum_{i \in \mathcal{B}}\big[-\big(\mathbf{w}^\top \tilde{\Phi}(\mathbf{z}_i) - \rho\big)\big]_+, \tag{3.30}$$

where $\mathcal{B}$ is a mini-batch of size 256 and $[\cdot]_+$ denotes the hinge function. SGD with momentum 0.9 and learning rate 10^{-2} runs for $E_{\text{SVM}} = 500$ epochs. After SGD converges, the offset ρ is reset to the ν-quantile of the training projections $\mathbf{w}^\top \tilde{\Phi}(\mathbf{z}_i)$, enforcing the KKT condition that exactly a fraction ν of training points lie outside the boundary regardless of SGD convergence quality.

The number of Nyström landmarks is set to $m = 300$; this value was chosen to balance approximation accuracy against memory cost on a single GPU, and was not tuned systematically.

Table 3.1: Hyperparameters of the Transformer bottleneck autoencoder and Nyström OC-SVM.

Parameter	Symbol	Value	Source
Sequence length	k	25	Poutré et al. [4]
Model dimension	d_{model}	64	Empirical
Attention heads	h	4	Empirical
Transformer layers	N	2	Empirical
Representation dimension	d_{rep}	128	Poutré et al. [4]
Training loss	—	MSE (Frobenius)	Poutré et al. [4]
Optimiser	—	Adam	Poutré et al. [4]
Learning rate	η	10^{-4}	Empirical
Max epochs	E	1,000	Empirical
Early stopping patience	P	20	Empirical
Gradient clipping	—	max norm 1.0	Empirical
OC-SVM kernel	—	RBF	Poutré et al. [4]
Outlier fraction	ν	0.01	Empirical
RBF bandwidth	γ	Median heuristic	Empirical
Nyström landmarks	m	300	Empirical
SGD learning rate	—	10^{-2}	Empirical
SGD momentum	—	0.9	Empirical
SGD epochs	E_{SVM}	500	Empirical
SGD batch size	—	256	Empirical

3.2.2 Probabilistic Neural Network

The PNN follows the framework of Fabre and Challet [1], who train a neural network to predict the conditional probability distribution of the next mid-price move given the current order flow state. Two quantities are derived from this prediction: the negative log-likelihood of the realised price move, which serves as a general anomaly score (events where the realised move is unlikely under the model receive high scores), and a spoofing gain that quantifies the profit a rational manipulator could extract from the current book state.

Spoofing gain. Before describing the network, we define the economic quantity that motivates it. Consider a manipulative agent who wants to sell q units. She simultaneously places a bona fide ask order at distance δ_a from the best ask and a non-bona-fide (spoof) bid order of size $Q \gg q$ at distance δ_b from the best bid. The spoofing gain is the reduction in expected execution cost achieved by the spoof order:

$$G_t = \mathbb{E}\left[C^a_{\text{spoof}}(0, \delta_a, 0, q) \mid \mathbf{x}^0_t\right] - \mathbb{E}\left[C^a_{\text{spoof}}(\delta_b, \delta_a, Q, q) \mid \mathbf{x}_t\right], \tag{3.31}$$

where $C^a_{\text{spoof}}(\delta_b, \delta_a, Q, q)$ is the net cost of executing the strategy (genuine ask order filled, spoof bid order potentially filled, residual inventory liquidated at market) and $\mathbf{x}^0_t$, $\mathbf{x}_t$ denote the feature vectors without and with the spoof order, respectively [1]. $G_t > 0$ means spoofing is profitable at time t.

Under the assumptions of Fabre and Challet [1] (no partial fills, execution triggered when the mid-price crosses the order price, mandatory liquidation of unintended inventory), the expected cost depends on four terms involving the probability of fill on each side and the conditional expectation of the price move given fill or no fill. Let s_t denote the bid-ask spread

at time t, and let $\varepsilon^+, \varepsilon^-$ denote the maker and taker fees. For a seller, the expected cost is:

$$\begin{aligned}\mathbb{E}\left[C^a_{\text{spoof}}\right] = &-P\big(\Delta p > \delta_a + \tfrac{s_t}{2}\big)\,(1-\varepsilon^+)\,q\,(p^a_{1,t} + \delta_a)\\ &+ P\big(\Delta p < -\delta_b - \tfrac{s_t}{2}\big)\,(1+\varepsilon^+)\,Q\,(p^b_{1,t} - \delta_b)\\ &- P\big(\Delta p \le \delta_a + \tfrac{s_t}{2}\big)\,(1-\varepsilon^-)\,q\,\big(p^b_{1,t} + \mathbb{E}[\Delta p \mid \Delta p \le \delta_a + \tfrac{s_t}{2}]\big)\\ &- P\big(\Delta p < -\delta_b - \tfrac{s_t}{2}\big)\,(1-\varepsilon^-)\,Q\,\big(p^b_{1,t} + \mathbb{E}[\Delta p \mid \Delta p < -\delta_b - \tfrac{s_t}{2}]\big),\end{aligned} \tag{3.32}$$

where the probabilities and conditional expectations are evaluated under the PNN-predicted distribution of Δp.

Architecture. The PNN takes as input the flattened feature vector $\mathbf{x}_t \in \mathbb{R}^{k \cdot d}$, where $k = 25$ is the sequence length and d the number of features. A single hidden layer with $d_h = 64$ units and ReLU activation maps the input to a hidden representation. A linear output layer produces three scalars $(\mu_{\text{raw}}, \sigma_{\text{raw}}, \alpha_{\text{raw}})$. The location parameter $\mu = \mu_{\text{raw}}$ and the skewness parameter $\alpha = \alpha_{\text{raw}}$ are unconstrained, while the scale parameter is made strictly positive by:

$$\sigma = \text{Softplus}(\sigma_{\text{raw}}) + \epsilon, \qquad \epsilon = 10^{-6}. \tag{3.33}$$

The output triple (μ, σ, α) parameterises a skew-normal distribution for the next-step mid-price log return $y_t = \ln(p_{\text{mid},t+1}/p_{\text{mid},t})$.

Loss function. The training objective is the negative log-likelihood of the skew-normal distribution, following Equation (20) of Fabre and Challet [1]. The skew-normal density is:

$$f(y \mid \mu, \sigma, \alpha) = \frac{2}{\sigma}\,\varphi\left(\frac{y-\mu}{\sigma}\right)\Phi\left(\alpha \cdot \frac{y-\mu}{\sigma}\right), \tag{3.34}$$

where φ is the standard normal density and Φ is the standard normal CDF. The per-batch loss is:

$$\mathcal{L}_{\text{PNN}} = -\frac{1}{|\mathcal{B}|}\sum_{i \in \mathcal{B}} \ln\Big(f(y_i \mid \mu_i, \sigma_i, \alpha_i) + 10^{-10}\Big). \tag{3.35}$$

Training procedure. The Adam optimiser [2] is used with a learning rate of $\eta = 10^{-4}$ and a batch size of 64. Training runs for up to $E = 1{,}000$ epochs per day with early stopping on the validation loss (patience $P = 20$). The model is trained sequentially across trading days within the continual learning protocol described in Chapter 4.

Anomaly scoring. Two anomaly scores are available. The primary score, used for spoofing detection, is the spoofing gain G_t defined in Equation (3.31): positive values flag events where spoofing would be profitable. The secondary score is the per-sample NLL $-\ln f(y_t \mid \mu_t, \sigma_t, \alpha_t)$: high values indicate that the realised price move was unlikely under the model, which may signal any form of anomalous activity, not only spoofing.

Divergences from Fabre and Challet. The following aspects of the implementation differ from the original paper.

1. **Data source.** Fabre and Challet [1] work with Level-3 cryptocurrency data (BTC-USD and ETH-USD on Coinbase), where each individual order is observed with its volume and distance of placement. Our dataset provides Level-2 equity snapshots (aggregate volume at 10 price levels for TOTF.PA, Euronext Paris). The Hawkes-inspired features are approximated from volume changes at unchanged price levels, as described in Section 3.1.6.

2. **Feature set.** The original paper uses 31 features (24 limit-order flow, 6 marketable-order flow, 1 spread). Our pipeline uses approximately 80 features from seven groups

(Section 3.1), including imbalance, dynamics, elasticity, volatility, event flow, OFI, and Hawkes features. The PNN receives the full feature set, flattened over the sequence window.

3. **Batch size.** Fabre and Challet use a batch size of $2^{12} = 4{,}096$. Our implementation uses a batch size of 64, constrained by GPU memory when the flattened input dimension is large ($k \cdot d \approx 2{,}000$).

4. **Learning rate and patience.** The paper uses a learning rate of 10^{-3} and early stopping patience of 100 epochs. Our implementation uses 10^{-4} and patience 20, tuned for the smaller and noisier equity dataset.

5. **Continual training.** The paper trains on a fixed dataset of 10^6 orders. Our pipeline trains sequentially across trading days, carrying over weights from the previous day.

Table 3.2: Hyperparameters of the Probabilistic Neural Network.

Parameter	Symbol	Value	Source
Input dimension	—	$k \cdot d$ (flattened)	Fabre and Challet [1]
Hidden units	d_h	64	Fabre and Challet [1]
Hidden activation	—	ReLU	Fabre and Challet [1]
Output distribution	—	Skew-normal (μ, σ, α)	Fabre and Challet [1]
Loss	—	Skew-normal NLL	Fabre and Challet [1]
Optimiser	—	Adam	Fabre and Challet [1]
Learning rate	η	10^{-4}	Empirical
Batch size	$\lvert\mathcal{B}\rvert$	64	Empirical
Max epochs	E	1,000	Fabre and Challet [1]
Early stopping patience	P	20	Empirical

3.2.3 Probabilistic Robust Autoencoder

The PRAE, introduced by Lindenbaum et al. [3], addresses the contamination problem that affects all reconstruction-based anomaly detectors: when training data contains anomalous events, a standard autoencoder learns to reconstruct those anomalies alongside normal patterns, blurring the boundary between the two. The PRAE solves this by introducing a learnable per-sample gate that controls each training point's contribution to the reconstruction loss. During training, the gates are pushed toward 1 (inlier) by a regularisation term but toward 0 (outlier) by high reconstruction error, so the autoencoder focuses on reconstructing clean data.

Backbone. The PRAE uses the same bottleneck Transformer autoencoder architecture as the detector in Section 3.2.1 (same d_{model}, h, N, and d_{rep}). The Transformer backbone is wrapped in the PRAE framework; the bottleneck representation is not extracted for a separate classifier but is used internally within the reconstruction pipeline.

Stochastic gates. Each training sample i is assigned a learnable gate parameter $\mu_i \in \mathbb{R}$, initialised to 0.5. During each forward pass, a stochastic gate is drawn:

$$z_i = \text{clamp}\big(\mu_i + \varepsilon_i,\ 0,\ 1\big), \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \tag{3.36}$$

where σ is a fixed noise scale. When μ_i drifts below zero, z_i is clamped to zero with high probability, effectively removing sample i from the reconstruction loss. When μ_i is large and positive, $z_i \approx 1$ and the sample contributes fully.

Loss function. Following the ℓ_1 relaxation of Lindenbaum et al. [3] (their Equation 5), the training objective is:

$$\mathcal{L}_{\text{PRAE}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} z_i \left\| \mathbf{S}_i - \hat{\mathbf{S}}_i \right\|^2 - \lambda \cdot \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbb{E}[z_i], \tag{3.37}$$

where the first term is a Monte Carlo estimate of the gated reconstruction error (using the sampled z_i) and the second term is the regulariser that encourages including more samples. The expected gate value $\mathbb{E}[z_i]$ is computed analytically from the truncated Gaussian distribution of z_i given μ_i and σ:

$$\mathbb{E}[z_i] = \frac{\sigma}{\sqrt{2\pi}} \left(e^{-\mu_i^2/(2\sigma^2)} - e^{-(1-\mu_i)^2/(2\sigma^2)} \right) + (\mu_i - 1)\, \Phi\left(\frac{1-\mu_i}{\sigma} \right) - \mu_i\, \Phi\left(\frac{-\mu_i}{\sigma} \right) + 1, \tag{3.38}$$

where Φ is the standard normal CDF. Using the analytical expectation rather than a Monte Carlo estimate provides smooth gradients for μ_i even when z_i is clamped to 0 or 1, which stabilises training.

The regularisation parameter $\lambda > 0$ controls the trade-off between reconstruction fidelity and robustness. A large λ pushes all gates toward 1, recovering a standard autoencoder. A small λ allows the model to exclude more samples, increasing robustness but potentially discarding useful training signal.

Per-sample reconstruction errors are clamped to 10^6 to prevent overflow: when a gate drifts to zero, the backbone receives no gradient for that sample, its reconstruction error can grow unboundedly, and the product $0 \cdot \infty$ produces NaN under IEEE 754 arithmetic.

Training. The Adam optimiser with learning rate $\eta = 10^{-4}$ and batch size 64 is used, with early stopping on the validation loss (patience $P = 20$). On each new training day, the gate vector $\boldsymbol{\mu}$ is re-initialised to 0.5 for all samples in the current day's training set, because $\boldsymbol{\mu}$ has one entry per training sample and the sample count changes across days. The backbone weights are carried over from the previous day.

Lambda Tuning

The PRAE paper [3] does not prescribe a specific λ selection procedure; it treats λ as a hyperparameter to be set based on a contamination-level estimate. Our implementation uses a two-stage approach.

On the first training day, an initial $\lambda_{\text{heuristic}}$ is computed from the average energy of the training data:

$$\lambda_{\text{heuristic}} = \overline{\|\mathbf{x}\|^2} \cdot k \cdot d, \tag{3.39}$$

where $\overline{\|\mathbf{x}\|^2}$ is the mean squared magnitude per element over a sample of training batches. A grid search is then run over the multiplicative factors $\{0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0\}$, training a fresh PRAE for 15 epochs for each candidate $\lambda = f \cdot \lambda_{\text{heuristic}}$. The factor that minimises the validation reconstruction loss (without gates) is selected.

On subsequent training days (day 2 onward), the backbone is already partially trained, so the raw-energy heuristic overestimates the reconstruction scale. The selected λ from day 1 is retained for all subsequent days without re-tuning.

Anomaly scoring. At inference, the stochastic gates are absent: the PRAE reduces to a standard autoencoder. The anomaly score for a test subsequence $\mathbf{S}_t$ is the per-sample squared reconstruction error:

$$s_t = \left\| \mathbf{S}_t - \hat{\mathbf{S}}_t \right\|^2. \tag{3.40}$$

High values indicate that the input deviates from the learned reconstruction manifold, which was fit to clean data thanks to the gating mechanism during training.

Table 3.3: Hyperparameters of the Probabilistic Robust Autoencoder.

Parameter	Symbol	Value	Source
Backbone	—	Bottleneck Transformer (Table 3.1)	Empirical
Gate noise scale	σ	0.5	Lindenbaum et al. [3]
Gate initialisation	μ_i	0.5	Empirical
λ selection	—	Grid search (7 factors $\times$ 15 epochs)	Empirical
Reconstruction clamp	—	10^6	Empirical
Optimiser	—	Adam	Empirical
Learning rate	η	10^{-4}	Empirical
Max epochs	E	1,000	Empirical
Early stopping patience	P	20	Empirical
Batch size	$\lvert\mathcal{B}\rvert$	64	Empirical

3.3 Anomaly Thresholding

3.3.1 Peak Over Threshold

3.3.2 SPOT and DSPOT

3.3.3 Rolling False Discovery Rate

3.4 Spoofing Gain Estimation

3.5 Interpretability

3.5.1 Integrated Gradients

3.5.2 Grouped Occlusion

3.6 Post-Hoc Analysis

3.6.1 Anomaly Clustering

3.6.2 Jump Analysis

Chapter 4

Data and Experimental Setup

4.1 Dataset Description

4.2 Preprocessing Pipeline

4.3 Training Protocol

4.3.1 Model Hyperparameters

4.4 Test Protocol

4.5 Compute Environment

Chapter 5

Results

5.1 Model Diagnostics

5.1.1 Transformer + OC-SVM

5.1.2 PNN

5.1.3 PRAE

5.2 Anomaly Detection on Test Data

5.2.1 Anomaly Rates by Model and Time Period

5.2.2 Score Time Series

5.2.3 Model Agreement

5.3 Threshold Comparison

5.4 Spoofing Gain Analysis

5.5 Sensitivity and Feature Attribution

5.5.1 Integrated Gradients: PNN and PRAE

5.5.2 Grouped Occlusion: Transformer + OC-SVM

5.5.3 Cross-Model Feature Importance

5.6 Anomaly Clustering

5.7 Jump Analysis

5.8 Cross-Year Robustness

Chapter 6

Discussion

6.1 Limitations

6.2 Comparison of Detection Approaches

6.3 Interpretation of Detected Anomalies

Chapter 7

Conclusion

7.1 Summary of Contributions

7.2 Future Work

Bibliography

[1] Timothée Fabre and Damien Challet. *Learning the Spoofability of Limit Order Books With Interpretable Probabilistic Neural Networks*. 2025. arXiv: 2504.15908 [q-fin.TR]. URL: https://arxiv.org/abs/2504.15908.

[2] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *arXiv preprint arXiv:1412.6980* (2014).

[3] Ofir Lindenbaum, Yariv Aizenbud, and Yuval Kluger. *Probabilistic Robust Autoencoders for Outlier Detection*. 2022. arXiv: 2110.00494 [cs.LG]. URL: https://arxiv.org/abs/2110.00494.

[4] Cédric Poutré, Didier Chételat, and Manuel Morales. "Deep Unsupervised Anomaly Detection in High-Frequency Markets". In: *The Journal of Finance and Data Science* 10 (Apr. 2024), p. 100129. DOI: 10.1016/j.jfds.2024.100129.

[5] Alban Siffer et al. "Anomaly Detection in Streams with Extreme Value Theory". In: *KDD 2017 - Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Halifax, Canada, Aug. 2017. DOI: 10.1145/3097983.3098144. URL: https://hal.science/hal-01640325.

[6] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. *Axiomatic Attribution for Deep Networks*. 2017. arXiv: 1703.01365 [cs.LG]. URL: https://arxiv.org/abs/1703.01365.

[7] Xuan Tao et al. *On Detecting Spoofing Strategies in High Frequency Trading*. 2020. arXiv: 2009.14818 [q-fin.TR]. URL: https://arxiv.org/abs/2009.14818.

[8] Ashish Vaswani et al. *Attention Is All You Need*. 2017. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

Appendix A

Feature Catalogue

Appendix B

Hyperparameter Tables

Appendix C

Additional Figures

C.1 Score Distributions per Test Day

C.2 LOB Snapshots for Selected Anomalies

C.3 Feature Evolution Around Anomalies

Appendix D

Threshold Method Details

Appendix E

Spoofing Gain Derivation